

第 2 卷 暴力、枚举、记忆化与部分分

考试速查：主查拿分底线：全排列、子集枚举、组合 DFS、BFS 状态搜索、剪枝、DFS 加 memo、折半枚举和大数据合法兜底。

Contents

第 2 卷 暴力、枚举、记忆化与部分分	1
考试速查定位	1
暴力与部分分快速目录	1
BRUTE-00: 部分分总策略	2
BRUTE-01: 复杂度与数据范围速查	4
BRUTE-02: 合法兜底输出	7
BRUTE-03: 全排列枚举	9
BRUTE-04: 组合 / 选不选 DFS	11
BRUTE-05: 子集枚举与子集的子集	13
BRUTE-06: 回溯与剪枝	15
BRUTE-07: 记忆化搜索总论	18
BRUTE-08: 数组 / vector 记忆化	21
BRUTE-09: map<tuple,...> 记忆化	23
BRUTE-10: unordered_map + 编码记忆化	25
BRUTE-11: BFS 状态搜索	28
真正的状态 BFS: 网格 + 钥匙 mask	30
BRUTE-12: 折半枚举	32
BRUTE-13: 小数据精确 + 大数据特判	35
BRUTE-14: 暴力版本到优化版本的提交策略	37
BRUTE-15: 暴力 / 记忆化常见坑	39

第 2 卷 暴力、枚举、记忆化与部分分

考试速查定位

第 2 卷 暴力、枚举、记忆化与部分分

- 这是第几卷：02。
- 主要查什么：主查拿分底线：全排列、子集枚举、组合 DFS、BFS 状态搜索、剪枝、DFS 加 memo、折半枚举和大数据合法兜底。
- 什么时候翻：不会正解、想先交小数据、DFS 能写但会超时翻。
- 题面关键词：枚举；全排列；子集；DFS；BFS 状态；剪枝；记忆化；折半；合法兜底输出。

自动由 BRUTE 模块重建。定位是先活下来，再用记忆化和剪枝涨分。

暴力与部分分快速目录

数据/场景	先翻模块
$n \leq 10$ 排列/顺序	BRUTE-03
$n \leq 20$ 子集/状态	BRUTE-05/07/08
$n \leq 40$ 子集和/选或不选	BRUTE-12
能写 DFS 但会重复	BRUTE-07/08/09/10
最少步数状态搜索	BRUTE-11
正解不会但要提交	BRUTE-13/14/15

BRUTE-00: 部分分总策略

模块编号: BRUTE-00

模块名称: 部分分总策略

标签: 部分分、提交策略、考场节奏、先活下来

一句话用途: 不会正解时, 先写一个合法且能过小数据的版本, 再逐步升级到剪枝、记忆化或正式算法。

题面触发词:

- 多个子任务 / 部分分 / 数据范围分档。
- $n \leq 10$ 、 $n \leq 20$ 、 $n \leq 40$ 、 $n \leq 2000$ 等明显分层。
- 正解想不清, 但能枚举所有选择。
- 题目允许多次提交, 且取最高分。

适用场景:

- 机考时间紧, 每题先保证有分。
- 题面很长, 正解模型一时匹配不上。
- 可以写出暴力、特判或小数据精确解。

什么时候用:

- 看到小数据子任务。
- 看到选择、排列、子集、路径、方案数等可以搜索的结构。
- 需要先提交一个可运行版本验证输入输出格式。

不要什么时候用:

- 输出格式还没确认。
- 题目是交互题或答案必须完全正确才给分。
- 暴力会在最小数据都跑不完。

复杂度:

版本	常见复杂度	能拿的数据
合法兜底	$O(1)$ 或 $O(n)$	格式分、特判分
暴力 DFS	$O(2^n)$ 、 $O(n!)$	$n \leq 10 \sim 25$
DFS + 剪枝	依剪枝而定	比暴力略大
记忆化搜索	$O(\text{状态数} * \text{转移数})$	中档分, 经常可过
正式算法	题目要求	冲满分

数据范围参考:

- $n \leq 10$: 全排列、指数搜索通常可试。
- $n \leq 20$: 子集枚举、状压、DFS + memo。
- $n \leq 40$: 折半枚举。
- $n \leq 2000$: $O(n^2)$ 可能可过。
- $n \leq 2e5$: 通常要 $O(n \log n)$ 或 $O(n)$, 暴力只拿小数据。

依赖的标准容器:

- vector
- string
- queue
- map

- unordered_map
- tuple

输入如何整理：

把输入先整理成最容易枚举的标准容器：

- 序列：vector<long long> a(n + 1)，默认 1-index。
- 图：暂时用邻接表 vector<vector<int>> g(n + 1)。
- 状态：尽量变成若干个整数参数，例如 i, rest, last, mask。

接口：

```
// 考场版本路线，不是固定函数。
// V0: legal_fallback()
// V1: brute_force()
// V2: brute_force_with_pruning()
// V3: memoized_search()
// V4: official_or_optimized_solution()
```

输出能力：

- 能输出合法格式。
- 能处理样例和小数据。
- 能在有时间时继续升级，不推翻前一版代码。

下游可接：

- BRUTE-06 回溯与剪枝。
- BRUTE-07 记忆化搜索总论。
- DP 卷的模型 DP。
- 图论卷的 BFS / Dijkstra / Topo。

可拼接模块：

- BRUTE-01 复杂度与数据范围速查。
- BRUTE-02 合法兜底输出。
- BRUTE-03 全排列枚举。
- BRUTE-04 组合/选不选 DFS。
- BRUTE-05 子集枚举。

模板代码：

注意：下面完整程序只是“先交小数据/部分分”的演示，以最大子集和为例。它只保证 $n \leq 20$ 的暴力分支是精确的；大数据分支输出兜底值，不代表正确答案，不能当作正解模板直接提交。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
const int MAXN = 200000 + 5;
static ll a[MAXN];

// V0: 题目允许任意合法解时使用。实际输出要按题意改。
void legal_fallback() {
    cout << 0 << '\n';
}

// V1: 小数据暴力，先保证思路和输入输出能跑。
ll brute_force() {
    if (n > 20) return 0; // 子集暴力只给小数据；大数据走合法兜底或优化版。
    ll ans = 0;
    // 示例：枚举子集，求最大子集和。实际题目把 sum/check 部分替换掉。
    for (int mask = 0; mask < (1 << n); mask++) {
        ll sum = 0;
        for (int i = 1; i <= n; i++) {
            if (mask & (1 << (i - 1))) sum += a[i];
        }
        ans = max(ans, sum);
    }
}
```

```

    }
    return ans;
}

// V2/V3: 后续把 brute_force 里的递归改成剪枝或 memo。
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    for (int i = 1; i <= n; i++) cin >> a[i];

    if (n == 0) {
        legal_fallback();
        return 0;
    }

    cout << brute_force() << '\n';
    return 0;
}

```

调用示例:

```

// 第一次提交: 只写 legal_fallback 或最小特判。
// 第二次提交: 补 brute_force。
// 第三次提交: 把 brute_force 中的 dfs 加 memo。
cout << brute_force() << '\n';

```

常见坑:

- 只顾写正解，最后一个版本都没提交。
- 小数据暴力忘记处理 $n = 0/1$ 。
- 输出格式错，导致合法兜底也没分。
- 多测时没有清空全局数组、memo、答案。
- 最大值题把初值设成 0，但答案可能是负数。

暴力/部分分替代:

- 不会做: 先输出题目允许的最小合法值。
- 会枚举: 写 2^n 或 $n!$ 。
- 会状态: 把 DFS 参数变成 memo key。
- 会模型: 再翻 DP / 图论 / 数据结构卷升级。

升级方向:

合法输出 -> 特判 -> 暴力 DFS -> 剪枝 -> 记忆化搜索 -> 表推 DP/正解

最小测试样例:

输入:

```

1
5

```

期望:

程序不崩溃，输出一行合法答案。

BRUTE-01: 复杂度与数据范围速查

模块编号: BRUTE-01

模块名称: 复杂度与数据范围速查

标签: 复杂度、数据范围、算法选择、时间预算

一句话用途: 用数据范围快速判断暴力、记忆化、折半或正式算法是否值得写。

题面触发词:

- $1 \leq n \leq \dots$
- T 组数据。
- 时间限制 $1s/2s/3s$ 。
- 子任务中出现小范围。

适用场景：

- 写代码前判断版本能拿哪一档分。
- TLE 后判断是算法复杂度问题还是常数问题。
- 在暴力和记忆化之间做取舍。

什么时候用：

- 每道题读完输入范围后立刻用。
- 每次新增一层循环或一维状态时用。

不要什么时候用：

- 不要只看 n ，还要看 T、边数 m 、值域 W 、状态转移数量。
- 不要把 $O(2^n)$ 当作永远不能写，小数据子任务正是它的目标。

复杂度：

本模块本身无运行复杂度；用于估算其他模块。

数据范围参考：

估计操作数	通常可行性
$\leq 1e6$	很稳
$1e7$	通常可过
$1e8$	C++ 勉强，常数要小
$> 1e8$	大概率 TLE

数据范围	常见可用方案
$n \leq 10$	$n!$ 、复杂回溯
$n \leq 20$	2^n 、状压、子集枚举
$n \leq 25$	DFS + 剪枝 / memo
$n \leq 40$	折半枚举 $2^{(n/2)}$
$n \leq 300$	$O(n^3)$
$n \leq 3000$	$O(n^2)$
$n \leq 2e5$	$O(n \log n)$ 或 $O(n)$

依赖的标准容器：

- 无强依赖。
- 估算状态时常配合 `vector`、`map`、`unordered_map`。

输入如何整理：

先在草稿纸上列出：

$n =$
 $m =$
 $T =$
 值域/容量 $W =$
 状态维度 $=$
 每个状态转移数 $=$

接口：

```
// 估算总操作数：状态数 * 每状态转移数 * 测试组数。
__int128 estimate_ops(long long states, long long transitions, long long T) {
    return (__int128)states * transitions * T;
}
```

输出能力：

- 判断某个版本是否适合提交。

- 确定优先写暴力、memo、折半还是换模型。

下游可接：

- 所有 BRUTE 模块。
- DP 卷的数据范围路由。
- 图论卷的最短路、连通性、拓扑模块。

可拼接模块：

- BRUTE-00 部分分总策略。
- BRUTE-07 记忆化搜索总论。
- BRUTE-12 折半枚举。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

string judge_ops(long double ops) {
    if (ops <= 1e6) return "very_safe";
    if (ops <= 1e7) return "safe";
    if (ops <= 1e8) return "maybe";
    return "danger";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    ll states, transitions, T;
    cin >> states >> transitions >> T;
    long double ops = (long double)states * transitions * T;
    cout << judge_ops(ops) << '\n';
    return 0;
}
```

调用示例：

```
// 0/1 背包 memo: n * W 个状态, 每个状态 2 个转移。
long double ops = (long double)n * W * 2;
if (ops <= 1e8) {
    // vector memo 可以尝试
}
```

常见坑：

- 忘记乘 T。
- 忘记每个状态还要枚举 k 或邻接边。
- map / unordered_map 常数比数组大很多。
- 2^{25} 看起来不大，但转移复杂时会爆。
- 递归深度和内存也要算，不只算时间。

暴力/部分分替代：

- 若 $n \leq 20$ 子任务存在，先写子集或 DFS。
- 若 $n \leq 40$ ，优先考虑折半。
- 若 $n * W$ 可承受，优先写 vector memo。
- 若状态不清晰，用 map<tuple,...> 先拿中档。

升级方向：

- $O(n!)$ -> 回溯剪枝 / 状压 DP。
- $O(2^n)$ -> 折半 / 状压 DP / 记忆化。
- $O(n^3)$ -> 前缀和 / 单调队列 / 数据结构优化。
- map memo -> vector memo 或编码 unordered_map。

最小测试样例：

输入：
1000 1000 1

输出：
very_safe

BRUTE-02：合法兜底输出

模块编号：BRUTE-02

模块名称：合法兜底输出

标签：兜底、输出格式、特判、避免零分

一句话用途：完全不会时，也先输出一个符合格式和约束的答案，避免因为空程序或格式错误直接 0 分。

题面触发词：

- 输出任意一个方案。
- 如果无解输出 -1。
- 输出最小/最大值。
- 多组数据，每组输出一行。
- 构造题、方案题、判断题。

适用场景：

- 正解完全没思路。
- 只会处理一部分特殊输入。
- 要先验证读入和输出格式。

什么时候用：

- 题目允许某些固定答案合法。
- 可以识别非常简单的无解/有解情况。
- 需要提交 Version 0。

不要什么时候用：

- 固定输出不一定合法，而且题目会严格校验方案。
- 输出答案必须和输入强相关，例如最短路、精确计数。
- 还没读懂输出行数 and 空格规则。

复杂度：

- 通常 $O(1)$ 或 $O(n)$ 。
- 多测为 $O(T * n)$ ，主要用于读完输入并输出格式正确。

数据范围参考：

- 任意范围都能作为 Version 0。
- 对大数据不会正解时，兜底负责“格式活着”，不是负责正确。

依赖的标准容器：

- `vector`
- `string`

输入如何整理：

即使只输出兜底，也尽量完整读入，避免下一组测试错位。

接口：

```
void solve_one_case();
```

输出能力：

- 判断题：输出 YES 或 NO 中更可能合法的一种。
- 数值题：输出题目允许范围内的数。
- 构造题：输出空方案、单元素方案或 -1。
- 最优化题：输出简单可行方案的值，而不是随便输出。

下游可接：

- BRUTE-13 小数据精确 + 大数据特判。
- BRUTE-14 提交版本路线。

可拼接模块：

- BRUTE-00 部分分总策略。
- BRUTE-01 复杂度与数据范围速查。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

void solve_one_case() {
    int n;
    cin >> n;
    vector<ll> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];

    // 下面只是兜底示例，考试时必须按题目输出格式修改。
    // 如果题目要求输出一个数，并且 0 在合法范围内：
    cout << 0 << '\n';
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T = 1;
    // 如果题目有多测，再打开这一行：
    // cin >> T;
    while (T--) solve_one_case();
    return 0;
}
```

调用示例：

```
// 构造题：无法构造时通常可以先输出 -1。
cout << -1 << '\n';

// 判断题：若空集/空操作通常合法，可以先输出 YES。
cout << "YES\n";

// 方案题：输出方案长度 0，前提是题目允许空方案。
cout << 0 << '\n';
```

常见坑：

- 只输出答案，不读完整输入，多测时后面错位。
- 题目要求 Yes/No，却输出 YES/NO；大小写要看题面。
- 构造题输出的方案长度和方案元素数量不一致。
- 图题输出边或点时越界。
- 最小值题随便输出 0，但答案可能必须至少为 1。

暴力/部分分替代：

- 能识别 $n == 1$ ：先写单点正确答案。
- 能识别全相同：写全相同特判。
- 能识别空操作：输出空方案。
- 能构造任意合法方案：先输出这个方案，再考虑优化值。

升级方向：

固定合法输出 -> 简单特判 -> 小数据暴力 -> 大数据特判 -> 正式算法

最小测试样例：

输入：

1

期望：

输出一行，且不崩溃。

BRUTE-03：全排列枚举

模块编号：BRUTE-03

模块名称：全排列枚举

标签：排列、DFS、next_permutation、暴力

一句话用途：当题目要求尝试所有顺序时，枚举 $1..n$ 或给定数组的所有排列。

题面触发词：

- 排列顺序。
- 访问顺序。
- 安排任务顺序。
- 每个元素恰好用一次。
- $n \leq 8/10/11$ 。

适用场景：

- 需要枚举所有访问顺序。
- TSP 小数据。
- 调度、排列计分、暴力找最优顺序。

什么时候用：

- $n \leq 10$ 且每个排列检查较快。
- 题目显式要求每个元素用一次。
- 可以通过剪枝提前停止无效排列。

不要什么时候用：

- $n \geq 12$ 且无强剪枝。
- 元素可重复选择，这时是 DFS 选序列，不是排列。
- 只关心集合不关心顺序，应改用组合或子集。

复杂度：

- $O(n! * \text{check_cost})$ 。
- DFS 版本空间 $O(n)$ 。

数据范围参考：

- $n \leq 9$ ：通常稳。
- $n = 10/11$ ：看 `check_cost` 和时限。
- $n \geq 12$ ：只适合部分分或强剪枝。

依赖的标准容器：

- `algorithm`

输入如何整理：

- 若是 $1..n$ ，用静态数组 `p[1..n]` 存。
- 若是给定数组，先排序再 `next_permutation`，可自动避免重复排列。

接口：

```
ll solve_by_next_permutation();
```

输出能力：

- 最大/最小排列得分。
- 找到任意满足条件的排列。
- 统计满足条件的排列数。

下游可接：

- BRUTE-06 回溯与剪枝。
- BRUTE-07 记忆化搜索总论。
- 状压 DP。

可拼接模块：

- BRUTE-01 复杂度速查。
- BRUTE-14 提交版本路线。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);

int n;
const int MAXN = 12 + 5;
int p[MAXN];

ll score_permutation() {
    ll score = 0;
    for (int i = 1; i <= n; i++) {
        score += 1LL * i * p[i];
    }
    return score;
}

ll solve_by_next_permutation() {
    sort(p + 1, p + n + 1);
    ll best = -LINF;
    do {
        best = max(best, score_permutation());
    } while (next_permutation(p + 1, p + n + 1));
    return best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    for (int i = 1; i <= n; i++) cin >> p[i];
    cout << solve_by_next_permutation() << '\n';
    return 0;
}
```

调用示例：

```
int p[4] = {0, 1, 2, 3}; // p[1..3]
sort(p + 1, p + 4);
do {
    // check p
} while (next_permutation(p + 1, p + 4));
```

常见坑：

- 忘记先 `sort`，导致漏掉字典序前面的排列。
- 给定数组有重复元素时，手写 DFS 可能重复枚举；`next_permutation` 更省心。
- $n!$ 爆炸，别对 $n = 15$ 硬跑。
- 最大值初值不能随便设成 0。

暴力/部分分替代：

- $n \leq 10$ ：全排列直接交。
- $n > 10$ ：只处理小数据，或加剪枝，或换状压/贪心/DP。

升级方向：

- 全排列 -> 回溯剪枝。
- 全排列 -> 状压 DP: `dp[mask][last]`。
- 全排列 -> BFS 状态搜索: 最少交换次数等问题。

最小测试样例:

输入:

3
1 2 3

输出:

14

BRUTE-04: 组合 / 选不选 DFS

模块编号: BRUTE-04

模块名称: 组合 / 选不选 DFS

标签: 组合、DFS、选不选、搜索树、子序列

一句话用途: 每个元素只考虑选或不选, 枚举所有组合或满足限制的选择集合。

题面触发词:

- 选若干个。
- 每个物品最多选一次。
- 从 n 个中选 k 个。
- 容量、预算、代价限制。
- $n \leq 20/30$ 。

适用场景:

- 枚举所有子集。
- 枚举大小为 k 的组合。
- 0/1 背包小数据暴力。
- 需要保留当前选择路径。

什么时候用:

- 顺序不重要, 只关心选了哪些元素。
- 每个元素最多一次。
- 可以根据剩余数量或当前代价剪枝。

不要什么时候用:

- 元素可以无限次选, 应转完全背包或递归枚举次数。
- 顺序有意义, 应使用排列或序列 DFS。
- n 很大且没有剪枝或 memo。

复杂度:

- 选/不选: $O(2^n)$ 。
- 选 k 个: $O(C(n, k))$ 。
- 递归空间: $O(n)$ 。

数据范围参考:

- $n \leq 25$: 常见可尝试。
- $n \leq 40$: 考虑折半枚举。
- $n * W$ 较小: 考虑记忆化或背包 DP。

依赖的标准容器:

- `vector`

输入如何整理:

- 物品属性用 1-index: `cost[i]`、`value[i]`。
- 当前选择可用 `vector<int> chosen`。

接口:

```
void dfs(int i, long long sum);
```

输出能力:

- 最大值 / 最小值。
- 方案计数。
- 输出一个合法组合。
- 精确枚举所有组合。

下游可接:

- BRUTE-06 回溯与剪枝。
- BRUTE-07 记忆化搜索。
- DP 卷 0/1 背包。

可拼接模块:

- BRUTE-01 复杂度速查。
- BRUTE-08 vector memo。
- BRUTE-12 折半枚举。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

const ll LINF = (1LL << 62);
int n, k;
vector<ll> a;
vector<int> chosen;
ll best = -LINF; // 如果必须选 k 个, 允许所有数都为负

void dfs_choose_k(int i, ll sum) {
    if ((int)chosen.size() == k) {
        best = max(best, sum);
        return;
    }
    if (i == n + 1) return;

    int need = k - (int)chosen.size();
    int left = n - i + 1;
    if (left < need) return;

    chosen.push_back(i);
    dfs_choose_k(i + 1, sum + a[i]);
    chosen.pop_back();

    dfs_choose_k(i + 1, sum);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> k;
    a.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    if (k < 0 || k > n) {
        cout << "-1\n";
        return 0;
    }

    dfs_choose_k(1, 0);
    cout << best << '\n';
}
```

```
    return 0;
}
```

调用示例:

```
// 从第 1 个元素开始枚举, 当前和为 0。
dfs_choose_k(1, 0);
```

常见坑:

- `chosen.push_back` 后忘记 `pop_back`。
- 结束条件顺序错, 导致选满 `k` 后还继续递归。
- `left < need` 剪枝写错, 漏答案。
- 组合不关心顺序, 不要在递归里从头重新枚举。

暴力/部分替代:

- `n <= 25`: 直接选/不选。
- `n <= 40`: 折半枚举两边结果再合并。
- 容量 `W` 小: 改成 `dfs(i, rest) + memo` 或背包 DP。

升级方向:

- 组合 DFS -> 加上下界/上界剪枝。
- 组合 DFS -> 记忆化 `dfs(i, rest)`。
- 组合 DFS -> 0/1 背包表推。

最小测试样例:

输入:

```
4 2
1 5 3 2
```

输出:

```
8
```

BRUTE-05: 子集枚举与子集的子集

模块编号: BRUTE-05

模块名称: 子集枚举与子集的子集

标签: bitmask、子集、状压、枚举子掩码

一句话用途: 当 `n <= 20` 且状态是集合时, 用二进制掩码枚举集合、子集或子集的子集。

题面触发词:

- 集合。
- 已选择/未选择。
- 访问过哪些点。
- `n <= 20`。
- 状态压缩、二进制。

适用场景:

- 枚举所有子集。
- 求每个集合的代价。
- 枚举 `mask` 的所有子集 `sub`。
- 状压 DP 的预处理或转移。

什么时候用:

- 元素数量不超过 20 到 22。
- 集合能用一个整数表示。
- 需要快速判断某元素是否已选。

不要什么时候用:

- `n >= 25` 且需要枚举全部 2^n 。
- 集合元素不是小整数编号, 需先压缩。

- 子集的子集总复杂度是 $O(3^n)$ ，不能误以为是 $O(2^n)$ 。

复杂度：

- 所有子集： $O(2^n * \text{check_cost})$ 。
- 所有 mask 的所有 sub：总计 $O(3^n)$ 。
- 空间：常见 $O(2^n)$ 。

数据范围参考：

- $n \leq 20$ ： 2^n 通常可用。
- $n \leq 16$ ： 3^n 可能可用。
- $n > 22$ ：优先考虑折半或其他模型。

依赖的标准容器：

- 全局静态数组。

输入如何整理：

- 元素编号保持 $1..n$ 。
- 第 i 个元素对应 mask 的第 $i-1$ 位。
- 字符串仍按 C++ 自然下标；普通数组不改 0-index。

接口：

```
for (int mask = 0; mask < (1 << n); mask++) {}
for (int sub = mask; sub; sub = (sub - 1) & mask) {}
```

输出能力：

- 子集最大/最小值。
- 方案计数。
- 状压 DP 预处理。
- 集合划分转移。

下游可接：

- 状压 DP。
- BRUTE-12 折半枚举。
- BRUTE-07 记忆化搜索。

可拼接模块：

- BRUTE-01 复杂度速查。
- BRUTE-10 unordered_map 编码 memo。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    if (n < 0) {
        cout << 0 << '\n';
        return 0;
    }
    if (n > 22) {
        // 本完整示例只覆盖 n<=22；大数据走合法兜底，至少保证有输出。
        ll x;
        for (int i = 1; i <= n; i++) cin >> x;
        cout << 0 << '\n';
        return 0;
    }
    static ll a[25];
    for (int i = 1; i <= n; i++) cin >> a[i];
```

```

int total = 1 << n;
static ll sum[1 << 22];
for (int mask = 1; mask < total; mask++) {
    int b = __builtin_ctz(mask);
    int prev = mask ^ (1 << b);
    sum[mask] = sum[prev] + a[b + 1];
}

ll best = 0;
for (int mask = 0; mask < total; mask++) {
    best = max(best, sum[mask]);
}

cout << best << '\n';
return 0;
}

```

调用示例:

```

// 枚举 mask 的所有子集, 包括 0。
for (int sub = mask; ; sub = (sub - 1) & mask) {
    // use sub
    if (sub == 0) break;
}

// 枚举非空子集。
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // use sub
}

```

常见坑:

- $1 \ll n$ 当 $n \geq 31$ 会溢出, 应使用 $1LL \ll n$, 但通常 n 不应这么大。
- 子集枚举包含空集时, 循环要手动在 $sub == 0$ 后 break。
- `__builtin_ctz(0)` 未定义, 必须保证 $mask \neq 0$ 。
- 业务元素编号保持 1-index; 只有取位时用 $i-1$ 或 $b+1$ 转回元素编号。

暴力/部分替代:

- $n \leq 20$: 直接子集枚举。
- $n \leq 40$: 折半枚举左右两半。
- 集合 + 最优值: 尝试 $dp[mask]$ 或 $dfs(mask) + memo$ 。

升级方向:

- 子集枚举 $\rightarrow$ 状压 DP。
- 子集的子集 $\rightarrow$ 集合划分 DP。
- $2^n \rightarrow$ 折半枚举。

最小测试样例:

输入:

```

3
1 -2 4

```

输出:

```

5

```

BRUTE-06: 回溯与剪枝

模块编号: BRUTE-06

模块名称: 回溯与剪枝

标签: DFS、回溯、剪枝、搜索优化

一句话用途：在枚举选择时及时撤销现场，并用明显不可能更优的条件提前返回。

题面触发词：

- 搜索所有方案。
- 选若干个满足限制。
- 最大/最小答案。
- 字典序方案。
- n 小但纯暴力可能超时。

适用场景：

- 需要维护当前路径。
- 每一步做选择，递归返回后要恢复。
- 可以估计“当前分支最多/最少还能达到什么”。

什么时候用：

- DFS 已经能写，但跑得慢。
- 有容量、数量、剩余收益、排序后上界等限制。
- 题目要求找任意方案，找到后可以停止。

不要什么时候用：

- 分支之间存在大量重复状态，此时优先 memo。
- 剪枝条件不确定正确性，可能剪掉答案。
- 状态是图最短步数，通常 BFS 更稳。

复杂度：

- 最坏仍可能是指数级。
- 实际复杂度依赖剪枝强度。
- 空间通常为递归深度 $O(n)$ 。

数据范围参考：

- $n \leq 25$ ：DFS + 剪枝可作为部分分。
- $n \leq 40$ ：若无强剪枝，优先折半。
- 分支因子小、限制强时，可以处理更大数据。

依赖的标准容器：

- vector
- algorithm

输入如何整理：

- 把物品按“更容易剪枝”的顺序排序，例如价值大、代价小、候选少的先枚举。
- 预处理后缀上界，例如 `suffix_sum[i]` 表示从 $i..n$ 全选最多还能加多少。

接口：

```
void dfs(int i, long long current_value, long long current_cost);
```

输出能力：

- 最大值 / 最小值。
- 任意可行方案。
- 方案计数。

下游可接：

- BRUTE-07 记忆化搜索。
- BRUTE-12 折半枚举。
- DP 卷背包/状压 DP。

可拼接模块：

- BRUTE-03 全排列枚举。
- BRUTE-04 组合 DFS。
- BRUTE-05 子集枚举。

模板代码：


```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
ll W;
vector<ll> cost, value, suffix_value;
ll best = 0;

void dfs(int i, ll used, ll got) {
    if (used > W) return; // 可行性剪枝
    if (i == n + 1) {
        best = max(best, got);
        return;
    }

    // 最优性剪枝: 即使后面全选也不可能超过当前 best。
    if (got + suffix_value[i] <= best) return;

    // 优先尝试价值大的选择, 有时能更早提高 best。
    dfs(i + 1, used + cost[i], got + value[i]);
    dfs(i + 1, used, got);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 2, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    suffix_value.assign(n + 3, 0);
    for (int i = n; i >= 1; i--) {
        suffix_value[i] = suffix_value[i + 1] + max(0LL, value[i]);
    }

    dfs(1, 0, 0);
    cout << best << '\n';
    return 0;
}

```

调用示例:

```

best = 0;
dfs(1, 0, 0);
cout << best << '\n';

```

常见坑:

- 修改 chosen、used、cnt 后没有恢复。
- 剪枝条件用了估计下界/上界但方向写反。
- 用 $got + suffix_value[i] \leq best$ 时, `suffix_value` 必须真的是乐观上界。
- 找任意方案时, 递归返回后没有停止, 导致输出被覆盖。
- 多测时 `best` 和路径没有清空。

暴力/部分分替代:

- 先写不剪枝 DFS。
- 再加可行性剪枝: 超容量、数量不够、越界。
- 再加最优性剪枝: 后面全选也不可能更优。
- 仍慢时考虑 memo 或折半。

升级方向:

- 回溯 -> 记忆化 `dfs(i, rest)`。

- 回溯 -> 0/1 背包。
- 回溯 -> 分支限界 / A*, 低优先级。

最小测试样例：

输入：

```
3 5
3 10
4 20
2 8
```

输出：

```
20
```

BRUTE-07：记忆化搜索总论

模块编号：BRUTE-07

模块名称：记忆化搜索总论

标签：记忆化、DFS、状态、DP 入口、核心章节

一句话用途：把暴力 DFS 中重复计算的相同状态缓存起来，用最小改动得到中档甚至满分版本。

题面触发词：

- 每一步有选择。
- 暴力 DFS 能写出来。
- 同一个局面会从不同路径到达。
- 表推 DP 循环顺序想不清。
- $n * W$ 、 $mask * last$ 、 $pos * tight * state$ 等状态数可估算。

适用场景：

- 背包、区间、树、DAG、数位、状压等 DP 的递归写法。
- 递归参数能完整描述后续问题。
- 状态总数远小于搜索树节点数。

什么时候用：

- 已经能写出 `dfs(...)`。
- `dfs` 的返回值只由参数决定。
- 相同参数组合会重复出现。
- 状态数量可承受。

不要什么时候用：

- `dfs` 返回值依赖没有写进参数的全局变量。
- 相同参数下，后续答案还会因为路径不同而不同。
- 状态有环且没有环检测。
- 状态几乎不重复，`memo` 只会增加常数。
- 递归深度可能爆栈且无法改写。

复杂度：

记忆化复杂度 = 状态数 * 每个状态的转移数

空间复杂度 = 状态数

数据范围参考：

- $n * W \leq 1e7$ ：数组/vector memo 可尝试。
- 状态数量 $\leq 1e5 \sim 1e6$ ：`map<tuple>` 可作为稳妥版。
- 状态数量较多且能安全编码：`unordered_map`。
- `mask` 状态通常要求 $n \leq 20$ 。

依赖的标准容器：

- `vector`
- `map`
- `unordered_map`

- tuple

输入如何整理：

- 把 DFS 参数尽量整理成整数: `i`, `rest`, `last`, `mask`, `cnt`。
- 若参数有负数, 用 OFFSET 平移, 或改用 `map<tuple,...>`。
- 若参数是集合, 用 `mask` 表示。

接口：

// 固定句式：

// `dfs(状态参数)` 返回: 从这个状态继续走, 能得到的答案。

```
long long dfs(int i, int rest);
```

输出能力：

- 最大值。
- 最小值。
- 方案数。
- 可行性。
- 也可保存选择用于还原方案。

下游可接：

- DP 卷: 把 DFS 参数变成 DP 下标。
- BRUTE-08 vector memo。
- BRUTE-09 map memo。
- BRUTE-10 unordered_map 编码 memo。

可拼接模块：

- BRUTE-04 组合 DFS。
- BRUTE-05 子集枚举。
- BRUTE-06 回溯剪枝。
- BRUTE-15 常见坑。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);

int n, W;
vector<int> cost;
vector<ll> value;
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;          // 1. 先判非法, 防止数组越界
    if (i == n + 1) return 0;          // 2. 再判终止

    if (vis[i][rest]) return memo[i][rest]; // 3. 查缓存
    vis[i][rest] = 1;

    ll ans = -LINF;
    ans = max(ans, dfs(i + 1, rest)); // 不选
    if (cost[i] <= rest) {
        ans = max(ans, value[i] + dfs(i + 1, rest - cost[i])); // 选
    }

    return memo[i][rest] = ans;        // 4. 存缓存
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    memo.assign(n + 2, vector<ll>(W + 1, 0));
    vis.assign(n + 2, vector<int>(W + 1, 0));

    cout << dfs(1, W) << '\n';
    return 0;
}

```

调用示例:

```

// 暴力版本:
// ll dfs(int i, int rest);
// 记忆化版本只加三件事:
// 1. memo/vis 容器
// 2. if (vis[state]) return memo[state]
// 3. return memo[state] = ans
cout << dfs(1, W) << '\n';

```

状态能否缓存判断:

1. dfs 参数是什么?
2. dfs 返回值是什么?
3. 给定这些参数后, 未来答案是否唯一?
4. 是否还依赖当前路径、已选集合、上一个元素、剩余次数、颜色、方向?
5. 如果依赖, 把它加入参数; 如果无法加入, 不要缓存。

可缓存例子:

dfs(i, rest) = 从第 i 个物品开始, 剩余容量 rest 的最大价值。
给定 i 和 rest 后, 前面怎么选不影响后面, 所以可缓存。

不可缓存例子:

dfs(i, sum) 里还用全局 vector<int> chosen 判断相邻冲突。
如果 chosen 没有进入参数, 相同 i 和 sum 可能有不同后续, 不能缓存。

返回值四件套:

最大值: ans = -LINF; 非法返回 -LINF; 转移用 max。
最小值: ans = LINF; 非法返回 LINF; 转移用 min。
方案数: ans = 0; 成功边界返回 1; 非法返回 0; 转移用加法取模。
可行性: ans = false; 成功边界返回 true; 非法返回 false; 转移用 ||。

有环状态风险:

普通记忆化默认状态依赖是无环的。如果 dfs(a) 可能还没算完又调用回 dfs(a), 只用 vis 会出错。

```

// 0 = 未访问, 1 = 正在访问, 2 = 已完成
vector<int> color;

bool dfs_cycle(int u) {
    if (color[u] == 1) return false; // 发现环, 按题意处理
    if (color[u] == 2) return true;
    color[u] = 1;
    // for (int v : g[u]) if (!dfs_cycle(v)) return false;
    color[u] = 2;
    return true;
}

```

常见坑:

- 非法状态晚于 memo 查询, 导致数组下标越界。
- 漏掉 last、mask、cnt 等影响未来的参数。
- 多测不清空 memo。
- 最大值题初始化为 0, 全负数时错。
- 计数题忘记取模。

- 有环状态直接递归，死循环。
- `vis=1` 表示“算完”，不要在状态仍在计算中就当成答案可用。

暴力/部分分替代：

- 没有重复状态：保留 DFS + 剪枝。
- 状态范围不清：先用 `map<tuple, ...>`。
- 状态范围清楚：换 `vector`。
- 表推顺序不会：保留记忆化提交。

升级方向：

暴力 DFS -> 记忆化 DFS -> 表推 DP -> 滚动数组/数据结构优化

最小测试样例：

输入：

```
3 5
3 10
4 20
2 8
```

输出：

```
20
```

BRUTE-08: 数组 / vector 记忆化

模块编号: BRUTE-08

模块名称: 数组 / vector 记忆化

标签: vector memo、数组 memo、状态范围明确、最快版本

一句话用途: 当每一维状态范围都清楚时，用 `vector` 或数组做最快、最稳的记忆化。

题面触发词：

- $0 \leq i \leq n$
- $0 \leq \text{rest} \leq W$
- $\text{mask} < 2^n$
- 维度小、边界明确。

适用场景：

- 背包类 `dfs(i, rest)`。
- 网格类 `dfs(x, y)`。
- 状压类 `dfs(mask, last)`。
- 数位 DP 中小范围状态。

什么时候用：

- 每个参数都能映射到非负整数下标。
- 总状态数能开内存。
- 追求比 `map` 更快。

不要什么时候用：

- 参数有大负数、大值域、字符串或复杂结构。
- 状态稀疏但范围巨大。
- 维度太多导致内存不可控。

复杂度：

- 时间: $O(\text{状态数} * \text{转移数})$ 。
- 空间: $O(\text{状态数})$ 。
- 查询/写入: $O(1)$ 。

数据范围参考：

- $n * W \leq 1e7$ 比较稳。
- $2^n * n$ 要求 $n \leq 20$ 左右。

- `vector<vector<vector<...>>>` 维度越多越要先算内存。

依赖的标准容器:

- `vector`

输入如何整理:

- 普通物品/序列下标统一用 $1..n$; 如果用 `vector`, 也开 $n + 1$ 。
- 若 `rest` 可为负, 先在函数开头拦截, 不要访问 `memo[i][rest]`。
- 若状态含负数, 例如 `diff in [-S,S]`, 使用 `diff + OFFSET`。

接口:

```
vector<vector<long long>> memo;
vector<vector<int>> vis;
long long dfs(int i, int rest);
```

输出能力:

- 最大值 / 最小值 / 计数 / 可行性。
- 可以额外存 `choice[i][rest]` 还原路径。

下游可接:

- DP 卷表推背包、网格、状压。
- BRUTE-14 提交版本路线。

可拼接模块:

- BRUTE-07 记忆化搜索总论。
- BRUTE-04 组合 DFS。
- BRUTE-05 子集枚举。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);

int n, W;
vector<int> cost;
vector<ll> value;
vector<vector<ll>> memo;
vector<vector<int>> vis;

ll dfs(int i, int rest) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    if (vis[i][rest]) return memo[i][rest];
    vis[i][rest] = 1;

    ll ans = dfs(i + 1, rest);
    ans = max(ans, value[i] + dfs(i + 1, rest - cost[i]));

    return memo[i][rest] = ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    memo.assign(n + 2, vector<ll>(W + 1, 0));
```

```
vis.assign(n + 2, vector<int>(W + 1, 0));

cout << dfs(1, W) << '\n';
return 0;
}
```

调用示例:

```
memo.assign(n + 2, vector<ll>(W + 1, 0));
vis.assign(n + 2, vector<int>(W + 1, 0));
cout << dfs(1, W) << '\n';
```

常见坑:

- `rest < 0` 之后才访问 `memo[i][rest]`, 直接越界。
- `memo` 初值和真实答案冲突, 所以建议单独用 `vis`。
- 多测时 `memo/vis` 没有重新 `assign`。
- `W` 很大时开二维数组 MLE。
- `vector<bool>` 有特殊实现, 不建议做通用 `memo` 标记。

暴力/部分替代:

- 状态范围太大: 改 `map<tuple,...>`。
- 只需小数据: 保留 DFS。
- `rest` 维很大但实际状态少: 用 `unordered_map` 或 `map`。

升级方向:

- `vector memo` -> 表推 DP。
- 二维 `memo` -> 滚动数组。
- `dfs(i, rest)` -> `dp[i][rest]` 或 `dp[rest]`。

最小测试样例:

输入:

```
2 3
2 5
3 7
```

输出:

```
7
```

BRUTE-09: `map<tuple,...>` 记忆化

模块编号: BRUTE-09

模块名称: `map<tuple,...>` 记忆化

标签: `map`、`tuple`、通用 `memo`、稳妥版

一句话用途: 状态维度复杂或范围不清时, 用 `map<tuple<...>, value>` 快速写出不容易错的记忆化。

题面触发词:

- 状态里有多个整数。
- 某些参数可能为负数。
- 状态范围不好开数组。
- 先追求写对, 不追求极限速度。

适用场景:

- `dfs(i, rest, last)`。
- `dfs(pos, balance, tight)`。
- `dfs(u, parent_state, used)`。
- 临场从暴力改 `memo`。

什么时候用:

- 不确定每维最大值。
- 想避免手写哈希和编码冲突。

- 状态数量中等。

不要什么时候用：

- 状态数极大，map 常数会导致 TLE。
- key 中含大对象，例如整个 vector，除非只是小数据部分分。
- 状态范围清楚且需要速度，应该用 vector。

复杂度：

- 每次查询/写入： $O(\log \text{ 状态数})$ 。
- 总时间： $O(\text{状态数} * \text{转移数} * \log \text{ 状态数})$ 。
- 空间： $O(\text{状态数})$ 。

数据范围参考：

- $1e5$ 级状态通常可用。
- $1e6$ 级状态要谨慎，可能慢。
- 适合“先交中档”的稳妥版本。

依赖的标准容器：

- map
- tuple
- vector

输入如何整理：

- DFS 参数尽量用 int / long long。
- key 的字段顺序和 DFS 参数顺序保持一致。
- 如果状态字段是 bool，也可以直接放进 tuple。

接口：

```
map<tuple<int,int,int>, long long> memo;
long long dfs(int i, int rest, int last);
```

输出能力：

- 最大值 / 最小值 / 计数 / 可行性。
- 可处理负数状态和稀疏状态。

下游可接：

- BRUTE-10 unordered_map 编码 memo。
- DP 卷表推转换。

可拼接模块：

- BRUTE-07 记忆化搜索总论。
- BRUTE-15 常见坑。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);

int n, W;
vector<int> cost;
vector<ll> value;
map<tuple<int, int, int>, ll> memo;

// last: 上一个选择的物品编号, 0 表示还没选。
ll dfs(int i, int rest, int last) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    auto key = make_tuple(i, rest, last);
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;

    ll ans = -LINF;
    for (int j = i; j <= n; j++) {
        int w = cost[j];
        ll v = value[j];
        ll res = dfs(j + 1, rest - w, j);
        if (res > ans) ans = res + v;
    }
    memo[key] = ans;
    return ans;
}
```



```

    ll ans = dfs(i + 1, rest, last);
    if (last == 0 || i - last >= 2) {
        ans = max(ans, value[i] + dfs(i + 1, rest - cost[i], i));
    }

    memo[key] = ans;
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    cout << dfs(1, W, 0) << '\n';
    return 0;
}

```

调用示例:

```

memo.clear();
ll ans = dfs(1, W, 0);

```

常见坑:

- 用 memo[key] 判断是否存在, 会把不存在的 key 插入进去。
- 多测没有 memo.clear()。
- key 漏字段, 导致错误缓存。
- map 过慢时要改 vector 或 unordered_map。
- tuple 字段类型不一致, 例如把 long long 强塞成 int 溢出。

暴力/部分替代:

- 先写 map<tuple>, 不必一开始想编码。
- 若 TLE, 再看每维范围, 改成 vector 或编码。
- 若状态数很小, map 就是最终版本。

升级方向:

- map<tuple> -> unordered_map<long long, value>。
- map<tuple> -> 多维 vector。
- dfs -> 表推 DP。

最小测试样例:

输入:

```

3 5
2 5
2 6
3 7

```

输出:

```

12

```

BRUTE-10: unordered_map + 编码记忆化

模块编号: BRUTE-10

模块名称: unordered_map + 编码记忆化

标签: 哈希、编码、稀疏状态、性能升级

一句话用途: 当状态范围较大但实际访问稀疏, 且能保证编码不冲突时, 用 unordered_map<long long, ...> 加速 memo。

题面触发词：

- map 版本能过样例但可能慢。
- 状态范围已知。
- 状态稀疏，不适合开大数组。
- 多维整数状态。

适用场景：

- dfs(i, rest, last) 中 rest 范围很大但访问少。
- BFS/DFS 状态需要哈希去重。
- mask 与小整数维度组合。

什么时候用：

- 每一维范围能明确写出来。
- 能设计无冲突编码。
- 需要比 map 更快的查询。

不要什么时候用：

- 不知道每一维范围。
- 维度有负数但没有平移。
- 编码可能冲突。
- 状态范围小且稠密，vector 更好。

复杂度：

- 平均查询/写入： $O(1)$ 。
- 最坏可能退化，但考试中通常可接受。
- 总时间： $O(\text{状态数} * \text{转移数})$ 平均。

数据范围参考：

- 状态数量 $1e5 \sim 1e6$ 可尝试。
- 若 unordered_map 超时，可以 reserve 和调低装载因子。

依赖的标准容器：

- unordered_map
- vector

输入如何整理：

- 把状态维度转为非负整数。
- 为每一维确定上界，例如 $i \leq n$ 、 $rest \leq W$ 、 $last \leq n$ 。
- 用乘法进制编码，比位移更不容易算错。

接口：

```
long long encode(int i, int rest, int last);
unordered_map<long long, long long> memo;
```

输出能力：

- 最大值 / 最小值 / 计数 / 可行性。
- 稀疏状态快速缓存。

下游可接：

- BFS 状态搜索去重。
- 状压 DP 稀疏优化。

可拼接模块：

- BRUTE-07 记忆化搜索总论。
- BRUTE-09 map memo。
- BRUTE-11 BFS 状态搜索。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll LINF = (1LL << 60);
```

```

int n, W;
vector<int> cost;
vector<ll> value;
unordered_map<ll, ll> memo;

// 要求:  $0 \leq i \leq n+1$ ,  $0 \leq rest \leq W$ ,  $0 \leq last \leq n$ .
ll encode(int i, int rest, int last) {
    ll base_rest = (ll)W + 1;
    ll base_last = (ll)n + 1;
    return ((ll)i * base_rest + rest) * base_last + last;
}

ll dfs(int i, int rest, int last) {
    if (rest < 0) return -LINF;
    if (i == n + 1) return 0;

    ll key = encode(i, rest, last);
    auto it = memo.find(key);
    if (it != memo.end()) return it->second;

    ll ans = dfs(i + 1, rest, last);
    if (last == 0 || i - last >= 2) {
        ans = max(ans, value[i] + dfs(i + 1, rest - cost[i], i));
    }

    memo[key] = ans;
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    cost.assign(n + 1, 0);
    value.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> cost[i] >> value[i];

    memo.reserve(200000);
    memo.max_load_factor(0.7);

    cout << dfs(1, W, 0) << '\n';
    return 0;
}

```

调用示例:

```

memo.clear();
memo.reserve(estimated_states * 2);
cout << dfs(1, W, 0) << '\n';

```

常见坑:

- 编码冲突: 两个不同状态算出同一个 key。
- 乘法编码溢出 long long。
- 状态有负数没有加 OFFSET。
- 多测中 W 或 n 变化, 旧 key 不能复用, 必须 clear()。
- 用 memo[key] 查存在性, 会插入默认值。

暴力/部分分替代:

- 不确定编码: 退回 map<tuple, ...>。
- 状态范围小: 用 vector。
- 状态没重复: 保留 DFS + 剪枝。

升级方向:

- `map<tuple>` -> 编码 `unordered_map`。
- 编码 `memo` -> `vector memo`。
- 哈希 DFS -> 表推 DP 或 BFS 去重。

最小测试样例:

输入:

```
3 5
2 5
2 6
3 7
```

输出:

```
12
```

BRUTE-11: BFS 状态搜索

模块编号: BRUTE-11

模块名称: BFS 状态搜索

标签: BFS、最少步数、状态图、去重

一句话用途: 当每一步代价相同、要求最少操作次数时, 把局面当作状态, 用 BFS 按层扩展。

题面触发词:

- 最少几步。
- 最少操作次数。
- 每次可以做若干种操作。
- 从初始状态变到目标状态。
- 状态数量不大。

适用场景:

- 字符串变换。
- 小拼图、小迷宫、开锁。
- 图上附带额外状态, 例如位置 + 钥匙集合。
- 每条边权都是 1。

什么时候用:

- 目标是最少步数。
- 每次操作代价相同。
- 状态可以编码并去重。

不要什么时候用:

- 边权不等, 应该用 Dijkstra。
- 状态转移有负权或收益, 不是最短步数。
- 状态数量不可控。
- DFS + memo 适合求最优值, 但 BFS 更适合最短层数。

复杂度:

- 时间: $O(\text{状态数} * \text{每状态转移数})$ 。
- 空间: $O(\text{状态数})$ 。

数据范围参考:

- 网格 $n*m$ 可控时可用。
- 位置 * mask 通常要求关键点数 ≤ 20 。
- 字符串状态若排列长度 ≤ 9 可尝试。

依赖的标准容器:

- `queue`
- `unordered_map`
- `map`

- vector
- string

输入如何整理:

- 先定义状态, 例如 (x, y, mask)。
- 能用整数编码就用整数; 复杂状态先用 string 或 tuple。
- 用 dist 记录是否访问和步数。

接口:

```
int bfs(State start, State target);
```

输出能力:

- 最少步数。
- 是否可达。
- 可通过 pre 数组还原路径。

下游可接:

- 图论卷 BFS / Dijkstra。
- BRUTE-10 unordered_map 编码 memo。
- 状压 DP。

可拼接模块:

- BRUTE-01 复杂度速查。
- BRUTE-05 子集枚举。
- DP-16 状压 DP。
- GRAPH-02 BFS。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

struct State {
    int x, y, mask;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n + 1);
    for (int i = 1; i <= n; i++) {
        string row;
        cin >> row;
        grid[i] = " " + row; // grid[i][1..m]
    }

    int sx = 1, sy = 1, tx = 1, ty = 1;
    bool hasS = false, hasT = false;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (grid[i][j] == 'S') sx = i, sy = j, hasS = true;
            if (grid[i][j] == 'T') tx = i, ty = j, hasT = true;
        }
    }
    if (!hasS || !hasT) {
        cout << -1 << '\n';
        return 0;
    }

    vector<vector<int>> dist(n + 1, vector<int>(m + 1, -1));
    queue<pair<int, int>> q;
```

```

q.push({sx, sy});
dist[sx][sy] = 0;

int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};

while (!q.empty()) {
    auto [x, y] = q.front();
    q.pop();
    for (int dir = 0; dir < 4; dir++) {
        int nx = x + dx[dir];
        int ny = y + dy[dir];
        if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
        if (grid[nx][ny] == '#') continue;
        if (dist[nx][ny] != -1) continue;
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}

cout << dist[tx][ty] << '\n';
return 0;
}

```

调用示例:

```

queue<State> q;
dist[start_key] = 0;
q.push(start);
while (!q.empty()) {
    State s = q.front();
    q.pop();
    // enumerate next states
}

```

真正的状态 BFS: 网格 + 钥匙 mask

题面常见: 网格里有钥匙 a,b,c 和门 A,B,C, 拿到对应钥匙才能过门。状态必须是 (x,y,mask), 因为同一个格子, 拿过哪些钥匙会影响后面能走哪些门。

```

#include <bits/stdc++.h>
using namespace std;

```

```

struct State {
    int x;
    int y;
    int mask;
};

```

```

int shortest_path_with_keys(vector<string>& grid, int n, int m) {
    const int KMAX = 10;
    const int SMAX = 1 << KMAX;
    static int dist[105][105][SMAX];
    if (n <= 0 || n > 104 || m <= 0 || m > 104) return -1;

    int sx = 1, sy = 1, tx = 1, ty = 1;
    bool hasS = false, hasT = false;
    int key_id[26];
    fill(key_id, key_id + 26, -1);
    int key_count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            char c = grid[i][j];
            if (c == 'S') sx = i, sy = j, hasS = true;
            if (c == 'T') tx = i, ty = j, hasT = true;

```

```

        if ('a' <= c && c <= 'z' && key_id[c - 'a'] == -1) {
            key_id[c - 'a'] = key_count++;
        }
    }
}

if (!hasS || !hasT) return -1;
if (key_count > KMAX) return -1; // 状态太多, 本模板只拿小数据/常见钥匙数
int full = 1 << key_count;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
        for (int mask = 0; mask < full; mask++) dist[i][j][mask] = -1;
    }
}

queue<State> q;

dist[sx][sy][0] = 0;
q.push({sx, sy, 0});

int dx[4] = {1, -1, 0, 0};
int dy[4] = {0, 0, 1, -1};

while (!q.empty()) {
    State cur = q.front();
    q.pop();

    if (cur.x == tx && cur.y == ty) {
        return dist[cur.x][cur.y][cur.mask];
    }

    for (int dir = 0; dir < 4; dir++) {
        int nx = cur.x + dx[dir];
        int ny = cur.y + dy[dir];
        int nmask = cur.mask;

        if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
        char c = grid[nx][ny];
        if (c == '#') continue;

        if ('A' <= c && c <= 'Z' && c != 'S' && c != 'T') {
            int need = key_id[c - 'A'];
            if (need == -1) continue;
            if (((cur.mask >> need) & 1) == 0) continue;
        }

        if ('a' <= c && c <= 'z') {
            nmask |= 1 << key_id[c - 'a'];
        }

        if (dist[nx][ny][nmask] != -1) continue;
        dist[nx][ny][nmask] = dist[cur.x][cur.y][cur.mask] + 1;
        q.push({nx, ny, nmask});
    }
}

return -1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;

```

```

cin >> n >> m;
vector<string> grid(n + 1);
for (int i = 1; i <= n; i++) {
    string row;
    cin >> row;
    grid[i] = " " + row; // grid[i][1..m]
}

cout << shortest_path_with_keys(grid, n, m) << '\n';
return 0;
}

```

如果钥匙种类太多， $n*m*2^{\text{key_count}}$ 会爆，只能拿小数据或换思路。

常见坑：

- 入队时不标记访问，导致同一状态重复入队。
- BFS 只适用于等权最短路，不适合边权不同。
- 状态编码漏掉 mask、方向、剩余资源等字段。
- 网格坐标越界。
- 目标可能不可达，要输出题目要求的无解值。
- 看到同一个 (x,y) 但钥匙不同，不能共用一个 $\text{dist}[x][y]$ 。

暴力/部分替代：

- 状态很复杂：先用 $\text{map}<\text{tuple}, \dots>$ 或 $\text{unordered_map}<\text{string}, \text{int}>$ 去重。
- 只会普通网格：先写无附加状态版本。
- 边权不同：先做小数据 BFS，后续升级 Dijkstra。

升级方向：

- BFS -> 双向 BFS。
- BFS -> Dijkstra。
- BFS 状态 (pos, mask) -> 状压 DP。
- 普通网格 BFS -> $\text{dist}[x][y][\text{mask}]$ 状态 BFS。

最小测试样例：

输入：

```

3 3
S..
.#.
..T

```

输出：

```

4

```

BRUTE-12: 折半枚举

模块编号：BRUTE-12

模块名称：折半枚举

标签：meet-in-the-middle、折半、子集和、二分

一句话用途：当 $n \leq 40$ 导致 2^n 太大时，把元素分成两半分别枚举，再排序/二分合并。

题面触发词：

- $n \leq 40$ 。
- 选若干个。
- 子集和。
- 不超过容量的最大值。
- 精确凑出某个和。

适用场景：

- 子集和。
- 0/1 选择但 n 在 30 到 44 左右。

- 需要统计两边组合满足某条件。

什么时候用：

- 直接 2^n 不行，但 $2^{(n/2)}$ 可行。
- 左右两半结果可以合并。
- 合并能用排序、二分、双指针或哈希。

不要什么时候用：

- 选择之间有强顺序依赖，不能简单拆两半。
- 状态需要复杂路径信息，左右无法合并。
- n 很大， $2^{(n/2)}$ 也不可行。

复杂度：

- 枚举： $O(2^{(n/2)})$ 。
- 排序： $O(2^{(n/2)} \log 2^{(n/2)})$ 。
- 合并：通常 $O(2^{(n/2)} \log 2^{(n/2)})$ 或双指针。

数据范围参考：

- $n \leq 40$ ：经典适用。
- $n \leq 44$ ：看时限和内存。
- $n \leq 50$ ：通常需要更强优化或其他算法。

依赖的标准容器：

- `vector`
- `algorithm`

输入如何整理：

- 把数组拆成 $[1, mid]$ 和 $[mid+1, n]$ 。
- 分别枚举两半所有子集和。
- 对右半排序，左半每个值二分找搭配。

接口：

```
vector<long long> enum_sums(int l, int r);
```

输出能力：

- 不超过 W 的最大子集和。
- 是否存在和为 `target`。
- 满足条件的方案数量。

下游可接：

- 二分。
- 双指针。
- 哈希计数。
- DP 卷背包优化思路。

可拼接模块：

- BRUTE-05 子集枚举。
- BRUTE-01 复杂度速查。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
ll W;
vector<ll> a;

vector<ll> enum_sums(int l, int r) {
    int len = r - l + 1;
    vector<ll> sums;
    long long total = 1LL << len;
    for (long long mask = 0; mask < total; mask++) {
```

```

    ll s = 0;
    for (int i = 0; i < len; i++) {
        if (mask & (1LL << i)) s += a[1 + i];
    }
    sums.push_back(s);
}
return sums;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> W;
    a.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    int mid = n / 2;
    vector<ll> L = enum_sums(1, mid);
    vector<ll> R = enum_sums(mid + 1, n);
    sort(R.begin(), R.end());

    ll best = 0;
    for (ll x : L) {
        if (x > W) continue;
        auto it = upper_bound(R.begin(), R.end(), W - x);
        if (it == R.begin()) {
            best = max(best, x);
        } else {
            --it;
            best = max(best, x + *it);
        }
    }

    cout << best << '\n';
    return 0;
}

```

调用示例:

```

vector<ll> left = enum_sums(1, n / 2);
vector<ll> right = enum_sums(n / 2 + 1, n);
sort(right.begin(), right.end());

```

常见坑:

- $1 \ll \text{len}$ 当 $\text{len} \geq 31$ 溢出; 折半后一般 $\text{len} \leq 22$ 。
- 右半为空时也要有和 0。
- `upper_bound` 返回 `begin()` 时不能直接 `--it`。
- 本页完整模板默认 $a[i] \geq 0$ 、 $W \geq 0$ 、空集合法, 因此可以跳过 $x > W$ 的左半和。
- 如果有负数, 不能用 `if (x > W) continue`, 因为右半负数可能把总和拉回合法; 此时删掉该剪枝并把 `best` 初始化成 `-LINF`。
- 统计方案数时要处理重复和。

暴力/部分分替代:

- $n \leq 24$: 直接枚举全部子集。
- $n \leq 40$: 折半枚举。
- 容量 W 小: 背包 DP 可能更好。

升级方向:

- 折半 + 二分 -> 折半 + 双指针。
- 折半枚举 -> 哈希计数。
- 折半 -> 正式 DP 或搜索剪枝。

最小测试样例:

输入:

4 10
2 4 8 9

输出：
10

BRUTE-13: 小数据精确 + 大数据特判

模块编号: BRUTE-13

模块名称: 小数据精确 + 大数据特判

标签: 部分分、特判、小数据暴力、大数据兜底

一句话用途: 对小范围输入写精确算法, 对大范围输入写合法输出或明显特判, 最大化部分分。

题面触发词:

- 子任务按数据范围给分。
- $n \leq 20$ 一档, $n \leq 2e5$ 一档。
- 有特殊性质: 全相等、链、树、无边、已排序。
- 大数据正解不会。

适用场景:

- 题目有明显分档。
- 小数据能暴力或 memo 精确解决。
- 大数据至少能处理几种简单结构。

什么时候用:

- 不确定满分算法。
- 可以用 if 分流不同范围。
- 特殊性质容易检测。

不要什么时候用:

- 特判会破坏正确的小数据逻辑。
- 大数据兜底输出不合法。
- 分流条件和题目子任务条件不一致。

复杂度:

- 小数据: 按所选精确算法, 例如 $O(2^n)$ 。
- 大数据特判: 通常 $O(n)$ 或 $O(n \log n)$ 。

数据范围参考:

- $n \leq 20$: 子集 / DFS / 状压。
- $n \leq 40$: 折半。
- 大数据: 检测全相等、单调、无边、树链等特殊形态。

依赖的标准容器:

- vector
- algorithm

输入如何整理:

- 先完整读入。
- 写几个布尔函数检测特殊性质: all_equal()、is_sorted()、is_small()。
- 按“最确定正确”的分支优先返回。

接口:

```
long long solve_small_exact();  
long long solve_large_special();
```

输出能力:

- 小数据精确答案。
- 大数据特殊情况精确答案。

- 其他情况合法兜底。

下游可接：

- BRUTE-02 合法兜底输出。
- BRUTE-14 提交版本路线。

可拼接模块：

- BRUTE-04 组合 DFS。
- BRUTE-05 子集枚举。
- BRUTE-12 折半枚举。

模板代码：

注意：本完整程序是“分流策略演示”，不是最大子集和通解。 $n \leq 20$ 精确， $n > 20$ 只处理全非负等特殊情况，其余输出合法兜底；真实题目必须按题意替换兜底分支。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
vector<ll> a;

bool all_non_negative() {
    for (int i = 1; i <= n; i++) {
        if (a[i] < 0) return false;
    }
    return true;
}

ll solve_small_exact() {
    ll best = 0;
    int total = 1 << n;
    for (int mask = 0; mask < total; mask++) {
        ll sum = 0;
        for (int i = 1; i <= n; i++) {
            if (mask & (1 << (i - 1))) sum += a[i];
        }
        best = max(best, sum);
    }
    return best;
}

ll solve_large_special() {
    if (all_non_negative()) {
        ll sum = 0;
        for (int i = 1; i <= n; i++) sum += a[i];
        return sum;
    }
    // 兜底：空集合法时答案至少为 0。实际题目要按题意修改。
    return 0;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    a.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) cin >> a[i];

    if (n <= 20) cout << solve_small_exact() << '\n';
    else cout << solve_large_special() << '\n';
    return 0;
}
```

```
}
```

调用示例:

```
if (n <= 20) {  
    cout << solve_small_exact() << '\n';  
} else if (all_non_negative()) {  
    cout << solve_large_special() << '\n';  
} else {  
    cout << 0 << '\n';  
}
```

常见坑:

- 小数据用 $1 < n$, 但 n 可能超过 30, 必须先判断。
- 特判条件不充分, 输出了错误答案。
- 兜底答案不合法。
- 分支里忘记多测清空变量。
- 特殊性质检测本身复杂度太高。

暴力/部分替代:

- 先只写 $n \leq 20$ 精确。
- 再加 $n \leq 40$ 折半。
- 再加全相等、全非负、全非正、已排序、无边等特判。

升级方向:

小数据 DFS $\rightarrow$ 小数据 memo $\rightarrow$ 折半 $\rightarrow$ 大数据特殊性质 $\rightarrow$ 正式算法

最小测试样例:

输入:

```
3  
-1 5 2
```

输出:

```
7
```

BRUTE-14: 暴力版本到优化版本的提交策略

模块编号: BRUTE-14

模块名称: 暴力版本到优化版本的提交策略

标签: 提交路线、版本管理、32 次提交、升级路径

一句话用途: 把一次题目作答拆成多个可提交版本, 避免一直憋满分算法却没有得分程序。

题面触发词:

- 每题多次提交。
- 取最高分。
- 数据范围分档。
- 样例已过但正解未完成。

适用场景:

- 所有题。
- 尤其是搜索、DP、构造、图论题。
- 正解想不清但部分版本可写。

什么时候用:

- 读完题后。
- 写出第一个能运行版本后。
- 每次准备大改代码前。

不要什么时候用:

- 不要为了版本路线引入大量无关代码。

- 不要在已 AC 的版本上盲目大改，建议另存草稿或小步改。

复杂度：

本模块是流程模块；各版本复杂度由具体算法决定。

数据范围参考：

Version	目标	常见复杂度
V0	合法输出/特判	$O(1)$ / $O(n)$
V1	暴力 DFS	$O(2^n)$ / $O(n!)$
V2	剪枝	指数但更快
V3	记忆化搜索	$O(\text{状态数} * \text{转移数})$
V4	正式 DP/图论/数据结构	满足题目范围

依赖的标准容器：

- 无固定依赖。
- 常配合 `vector`、`map`、`unordered_map`。

输入如何整理：

- 从 V0 开始就写好完整读入。
- 后续版本只替换 `solve()` 里的核心算法。
- 多测框架一开始写对。

接口：

```
void solve();
```

输出能力：

- 每个版本都能独立提交。
- 每次升级都保留上一版思路，便于回退到脑内稳定版本。

下游可接：

- 全部算法卷。

可拼接模块：

- BRUTE-00 部分分总策略。
- BRUTE-02 合法兜底输出。
- BRUTE-07 记忆化搜索总论。
- BRUTE-13 小数据精确 + 大数据特判。

模板代码：

注意：本完整程序演示“V0/V1/V2 逐步提交路线”。`solve_v0()` 只是占位兜底，只有题目允许这种合法输出时才可用于大数据；否则必须继续升级到正解或更强部分分。

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int n;
vector<ll> a;

ll solve_v0() {
    return 0; // 按题意改成合法兜底
}

ll solve_v1_bruteforce() {
    if (n > 20) return solve_v0();
    ll best = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        ll sum = 0;
        for (int i = 1; i <= n; i++) {
            if (mask & (1 << (i - 1))) sum += a[i];
        }
    }
}
```

```

        best = max(best, sum);
    }
    return best;
}

ll solve() {
    return solve_v1_bruteforce();
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n;
    a.assign(n + 1, 0); // 动态写法也保留 a[1..n]
    for (int i = 1; i <= n; i++) cin >> a[i];

    cout << solve() << '\n';
    return 0;
}

```

调用示例:

```

// 第一次提交:
cout << solve_v0() << '\n';

// 第二次提交:
cout << solve_v1_bruteforce() << '\n';

// 第三次提交:
cout << solve_v3_memo() << '\n';

```

常见坑:

- 没有先交任何版本。
- 大改时把读入和输出格式改坏。
- 多测版本和单测版本切换错误。
- 在小数据暴力里没有限制 n , 大数据直接 TLE。
- 提交前没有跑最小样例和边界样例。

暴力/部分分替代:

- V0: 合法输出。
- V1: 暴力枚举。
- V2: 剪枝。
- V3: memo。
- V4: 正式算法。

升级方向:

读题 10 分钟内: V0/V1 至少一个可提交

中段时间: V2/V3 拿中档

最后时间: V4 或补特判, 不要推倒重来

最小测试样例:

输入:

```

2
-5 3

```

输出:

```

3

```

BRUTE-15: 暴力 / 记忆化常见坑

模块编号: BRUTE-15

模块名称：暴力 / 记忆化常见坑

标签：坑表、调试、WA、TLE、RE

一句话用途：提交前按清单检查搜索、回溯、记忆化和部分分代码中最容易出错的地方。

题面触发词：

- 样例过了但小极端错。
- RE / TLE / WA。
- 多测。
- 状态复杂。

适用场景：

- 每次提交前。
- 从暴力升级到 memo 后。
- 出现死递归、越界、答案不稳定时。

什么时候用：

- 写完 DFS。
- 加完 memo。
- 加完剪枝。
- 改多测。

不要什么时候用：

- 不要只靠坑表替代样例测试。
- 不要为了修坑盲目改变算法含义。

复杂度：

本模块是检查清单，无运行复杂度。

数据范围参考：

- 所有范围适用。
- 越是小数据正确性，越应该先用坑表查。

依赖的标准容器：

- vector
- map
- unordered_map
- queue

输入如何整理：

- 多测题：所有全局变量必须在每组重置。
- 状态题：把状态字段写在注释里，逐一核对是否进入 key。

接口：

提交前检查清单。

输出能力：

- 发现潜在 WA / RE / TLE。
- 指导构造最小反例。

下游可接：

- 05_review 质检。
- 第 7 卷调试与反例。

可拼接模块：

- BRUTE-07 记忆化搜索总论。
- BRUTE-08 vector memo。
- BRUTE-09 map memo。
- BRUTE-10 unordered_map 编码 memo。
- BRUTE-11 BFS 状态搜索。

模板代码：


```

// 这是一段检查顺序，不是完整算法。
// 1. 非法状态是否最先返回？
// if (rest < 0) return -LINF;
//
// 2. 终止状态是否正确？
// if (i == n + 1) return 0;
//
// 3. memo key 是否包含所有影响未来的字段？
// auto key = make_tuple(i, rest, last, mask);
//
// 4. 多测是否清空？
// memo.clear();
// vis.assign(...);
//
// 5. 初值是否适合题目类型？
// max: -LINF, min: LINF, count: 0, feasible: false

```

调用示例：

```

// 最大值题，答案可能为负：
ll ans = -LINF;

```

// 计数题：

```

ans = (ans + dfs(next_state)) % MOD;

```

// 可行性题：

```

if (dfs(next_state)) return memo[state] = true;

```

常见坑：

- 状态遗漏：last、mask、cnt、tight、leading、剩余资源、当前位置方向。
- 非法状态访问数组：memo[i][rest] 前没有判断 rest < 0。
- 初值错误：最大值题初值设 0，全负答案错。
- 多测未清空：上一组的 memo 影响下一组。
- 有环状态：递归回正在计算的状态。
- 剪枝错误：剪掉可能更优的分支。
- unordered_map 编码冲突。
- 1 << n 溢出。
- BFS 入队后不立刻标记，重复入队。
- 递归太深爆栈。

暴力/部分分替代：

- 如果 memo 出错，先退回暴力 DFS 对拍小数据。
- 如果 vector 越界，先改 map<tuple> 验证状态。
- 如果 unordered_map 怀疑编码错，退回 map<tuple>。
- 如果剪枝怀疑错，先关掉剪枝。

升级方向：

- 加断言检查范围。
- 写小数据暴力与 memo 对拍。
- 把递归改表推或 BFS。
- 把危险编码改为 tuple 稳妥版。

最小测试样例：

建议至少手测：

1. n = 1
2. 容量/资源 = 0
3. 全部选择非法
4. 全部选择合法
5. 答案为负数
6. 有重复状态的小例子